Fuzzing for Web Browser Under-Invalidation Bugs

Anonymous Submission

***Abstract*—Fuzzers are widely used to find security-related bugs in web browser parsers and media engines through crash testing, and to discover cross-browser and cross-version regressions via differential testing. We introduce a new type of browser correctness bug, under-invalidation, which occurs when a browser incorrectly updates its internal state after a style update. Under-invalidation bugs allow for *self-differential* testing, which tests a browser rendering engine against itself, leading to particularly clean fuzzing results.**

To find under-invalidation bugs efficiently, we develop LQC, a fuzzer that rapidly generates candidates web pages, tests them in a real browser via self-differential testing, and then minimizes and clusters any pages that witness under-invalidation bugs. In our evaluation, LQC is able to reproduce 7 human-submitted bugs mined from the Mozilla bug tracker with self-differential testing. LQC is also able to test hundreds of fuzzer candidates per minute in both Chrome and Firefox, finding dozens of bug candidates in each browsers (79 in Chrome and 125 in Firefox). Furthermore, LQC's bug minimization and clustering passes reduce the number of style elements by 62.60% and 99.32% relative to minimized reports, all while requiring only 14.16% percent of the runtime. Finally, we show that LQC finds 16 novel bugs, across both browsers, that were reported to and accepted by the developers, with 7 already fixed.

***Index Terms*—Browser fuzzing, layout invalidation, under-invalidation, self-differential testing**

I. Introduction

Web browsers are central to modern computing, hosting applications that correspond to over **80%** of daily work in some organizations [1]. Browser makers therefore invest heavily in browser correctness, [**?**], [**?**], [**?**], [**?**], [**?**], [**?**], including through fuzzing [**?**], which uses a large number of randomly generated inputs to identify bugs in the browser.

Browser developers use fuzzers to find both security and correctness bugs. On the security side, fuzzers such as **[TODO: XYZ]** [**?**], [**?**] detect crashes in parsers, network libraries, and media engines, which are both exposed to untrusted input and must maximize performance. On the correctness side, fuzzers such as R2Z2 [2], Janus [3] and Metamong [4] use screenshots to detect browser rendering inconsistencies between different versions of the same browser, or between renderings of the different pages in the same browser, that may indicate a regression or rendering bug. However, all of these approaches struggle to precisely identify *under-invalidation bugs*, a subtle yet critical type of correctness bug in browser layout engines.

Invalidation refers to the process by which the browser updates internal data structures in response to style changes [**?**]. Since style changes are frequent (often every frame, 60 or even 120 times per second, in response to animations), substantial research has been devoted to designing efficient invalidation procedures [**?**], [**?**]. Generally, browsers attempt to update *as little* of their internal data structures as possible; however, updating *too little* of this internal state leads to an incorrect rendering that mixes fresh (updated) and stale (not updated) data. Updating exactly the right set of fields requires reasoning about transitive dependencies between layout algorithms, reasoning that is difficult enough that public bug trackers for Chrome, Firefox, and Safari's layout engines contain numerous under-invalidation bugs [5]–[10]. Modern invalidation algorithms like LayoutNG [11] are purposefully designed to reduce the risk of under-invalidation, while some promising algorithms [12] have failed precisely because under-invalidation bugs proved difficult to find and fix.

To combat these issues, this paper introduces a new oracle that detects under-invalidation bugs via *self-differential testing*, where either a large or small change results in the *same* web page, and where differences thus indicate under-invalidation, and validate this oracle on bugs mined from real-world browser bug databases. We extend this basic idea into LQC, a fuzzer that uses self-differential testing to detect under-invalidation bugs using random generation and minimization of complex web pages and web page changes, and leveraging automated bug clustering to reduce the bug review burden. We demonstrate that LQC is effective at both rapidly finding under-invalidation bugs (79 in Chrome and 125 in Firefox) and minimizing and clustering them, reducing review load by 99.74%. 16 bugs found by LQC were reported to Chrome and Firefox, all accepted by developers and 7 already fixed. LQC is now used as part of Mozilla's fuzzing workflow to find under-invalidation bugs during development.

This paper makes the following contributions:

- A new class of bugs, under-invalidation bugs, that can be efficiently detected using *self-differential* testing;
- LQC, a targeted fuzzer for under-invalidation bugs that generates web pages and web page changes, detects under-invalidation bugs with self-different testing, and minimizes and clusters the resulting bugs to reduce review load.
- An evaluation of LQC and its major components on both Chrome and Firefox, measuring bug discovery, minimization, and clustering with both direct and ablation studies.

We release LQC together with evaluation artifacts, test cases, and benchmarks at the following URL: **[TODO: URL]**.

II. Background, Related Work, and Motivation

This section introduces web browsers and layout, invalidation and under-invalidation bugs, and the challenges of fuzzing browser layout engines.

A. Web Browsers

Web browsers are among the most widely used and economically important software systems. A browser must load applications and content over the network; execute applications and react to user input; and render the application user interface to the screen. The rendering step is performed by the browser's *rendering engine* [**?**]; Fig. 1 is a high-level diagram of this rendering pipeline. The main production browser engines are Chrome's Blink engine [13], Firefox's Gecko engine [14], and Safari's WebKit engine [15], which power both the named browsers and many others, such as Microsoft Edge, Samsung Browser, and others [**?**]. These engines require substantial and sustained investment; for example, Open Web Advocacy estimates that Google provides roughly 90% of Blink development funding and invests around $1 billion annually in the web platform [16].

This paper focuses on *layout*, which involves computing the size and position of every web page element. Layout is one of the most complex parts of the rendering pipeline [**?**], and, because browser rendering is performed after every user interaction, it must be highly optimized [**?**], [17], [18] Rendering engines achieve high performance via *invalidation*: most user interactions, animations, or APIs only affect a part of the web application's user interface, and the browser attempts to re-render only that portion of the page, re-using prior rendering results for the rest [19]. This leads to large speed-ups [**?**], but risks two kinds of bugs [17], [20]. An *under-invalidation bug* occurs when the browser fails to recompute changed portions of the user interface, causing a partially stale interface to be shown the user, which can confuse the user or even break application functionality. An over-invalidation bug, by contrast, occurs when the browser recomputes too much of the user interface; this can slow rendering but doesn't cause correctness issues and isn't a focus of this paper [11].

Under-invalidation bugs often require a specific set of HTML elements, a specific set of CSS properties, and a set of JavaScript-triggered style updates to appear. For example, Chrome bug #40724799, found with LQC and since fixed by the Chrome developers, occurred only when an application had three nested elements, with the outermost having a `vertical-lr` writing style and the innermost having a `horizontal-tb` writing style, and the middle one having a `inline-grid` display style, and only when the application then set the white-space style of the middle element to `break-spaces` and back. Chrome would then require four rendering cycles to correctly compute the size and position of the middle element. Needless to say, constructing such an unlikely sequence of page elements, styles, and changes to those styles would be difficult for a manual testing approach.

[TODO: Put these cites somewhere if we need them: [21]–[23].]

B. Browser Fuzzing

Fuzzers have a long history in browser testing, especially due to browsers' exposure to untrusted data over the network. JavaScript-engine fuzzers such as CodeAlchemist and DIE generate complex programs to expose crashes and security vulnerabilities [24], [25]. Grammar-based fuzzers such as Domato [**?**], by contrast, generate structured inputs to exercise deeper portions of the browser pipeline, while other browser fuzzers target specialized attack surfaces such as WebGL or WebAssembly [26], [27]. These approaches focus on detecting crashes, which often indicate a security vulnerability. For example, RTFuzz mutates render-tree state, similar to LQC, but primarily searches for crashes [28].

A newer collection of fuzzers instead target rendering correctness. For example, R2Z2, Metamong, Janus [2], [3] generate structured web pages and compare their differential rendering either across browsers or across browser versions. This differential testing approach can detect cross-browser compatibility issues, but are challenging to scale because different browsers or versions can differ for legitimate reasons, including different platform libraries, different user interface elements, or different supported features, among others. Under-invalidation bugs are especially ill-suited to a differential testing approach, because browser implement different invalidation algorithms. On the other hand, directly testing a browser layout against a known-good or reference implementation is not possible, because no browser is known to generate correct output in all cases and no reference implementation exists.

A fuzzer for layout under-invalidation bugs is needed. Existing browser fuzzers, which focus on differential testing, cannot easily find under-invalidation bugs. LQC uses self-differential testing instead.

III. Challenges and Solutions

To address the limitations of conventional browser fuzzing for layout invalidation correctness, we introduce LQC: a system for finding, minimizing, and grouping under-invalidation bugs in modern web browser layout engines. In the following, we outline core challenges that shape LQC's design, along with their solutions aimed at effectively fuzzing browser layout invalidation.

A. Challenge 1: Defining a Correctness Oracle

A central challenge in testing layout invalidation is that there is no obvious external ground truth for the correct layout of a test case. Cross-browser comparisons can expose rendering differences, but those differences do not necessarily identify an invalidation bug because browsers may make different implementation, feature-support, or rendering choices. Crash-based or assertion-based fuzzing is also insufficient because an under-invalidation bug can produce a visually incorrect layout while the browser continues running normally. The difficult part is therefore not only generating a page that changes layout, but also deciding whether the browser reused stale layout information after that change.

No external ground truth: A generated HTML and CSS test case does not come with a known correct layout. Comparing against another browser can identify that two imple-

Fig. 1. A high-level diagram of the browser rendering pipeline. Of these steps, layout is most complex, as it instantiates the complex semantics of the CSS standard. The pipeline is rerun on every user interaction or animation, and invalidation is the main way browsers achieve high performance. Image reused with permission.

mentations disagree, but it cannot determine which browser is wrong or whether the difference is caused by invalidation. LQC therefore needs an oracle that does not depend on a separate browser or reference implementation.

Non-crashing failures: Under-invalidation bugs often do not crash the browser or violate an explicit assertion. Instead, the browser may continue executing while displaying stale geometry. This makes conventional crash-based fuzzing insufficient, because the fuzzer needs to detect a semantic layout mismatch rather than a process failure.

Path-dependent correctness: Under-invalidation depends on an execution path, not just a final page state. The same HTML and CSS may be correct when the browser computes layout from scratch, but incorrect when the browser reaches that state through an incremental update. A useful oracle must therefore compare two ways of reaching the same final page state: one through the incremental layout version and one through a refreshed layout computation.

Solution 1: LQC addresses this challenge using a self-differential testing oracle. For each test case, LQC compares the browser's incremental layout version after a style update against a refreshed layout version to identify a bug.

B. Challenge 2: Triggering Under-Invalidation Behavior

Valid HTML and CSS are not necessarily useful for testing layout invalidation. Under-invalidation occurs only when a page first establishes a layout state, then receives a meaningful style update, and the browser fails to recompute geometry affected by that update. An unconstrained fuzzer can produce many accepted pages without exercising these incremental-layout paths. LQC therefore biases generation toward CSS properties and mutations that are likely to change layout, so that each test is more likely to trigger the browser's invalidation logic.

Display: The `display` property is a useful target because it determines the layout mode used for an element and changes how its children participate in layout. Changing `display` can move an element between block, inline, flex, grid, table, or hidden states, each of which requires different layout behavior. These changes can affect both the modified element and its descendants, they provide strong opportunities to expose missed invalidation decisions in incremental layout engines [11], [19].

Grid: Grid layout creates many invalidation opportunities because it introduces relationships between a container and its children. Changing grid templates, gaps, placement rules, or auto-flow can reposition multiple children from a single style update. This makes grid a strong target for LQC because a missed invalidation can leave incorrect child positions or incorrect container geometry. Such dependent geometry is precisely the kind of cached result that incremental-layout implementations must invalidate correctly [19], [29].

Flow-root: The `display:flow-root` value is important because it establishes a new block formatting context and changes the boundary at which surrounding layout is resolved. Mutating it can therefore change both the target box and the geometry of neighboring content, making it useful for exercising invalidation propagation [19].

Width and height: Width and height changes directly affect an element's box geometry. When an element changes size, the browser may need to recompute that element, its descendants, siblings, and sometimes ancestors. These mutations are useful because layout engines often appears as an obvious mismatch in element size or position.

Percentage and pixel sizes: Percentage sizes test parent-dependent layout behavior because the final size of an element depends on its containing block. Pixel sizes create more direct change to the geometry possibly causing conflict issues with surrounding elements. Using both lets LQC exercise dependent layout constraints to produce failures that are easy to observe and reproduce. General browser fuzzers need not preserve such a parent-dependent update relationship, because their target oracles are typically crashes, security violations, or externally observed output differences [2], [24], [25], [30].

Transforms: Transform changes affect visual position and reported geometry. They provide a controlled update whose effect can be checked through the geometry oracle while interacting with containing blocks and descendant coordinates. This makes them useful for distinguishing a stale incremental result from a freshly computed final layout.

Noise reduction: LQC disables properties such as `writing-mode`, `content-visibility`, padding, and table display values because they are less likely to produce under-invalidation bugs. Instead, these properties often add unnecessary complexity to the test cases. Removing them keeps the test cases focused on mutations that are more likely to expose missed invalidation decisions and helps produce smaller, clearer bug reports. This focus is important because unconstrained random inputs can make an observed rendering difference hard to attribute to one invalidation decision [2]–[4].

Solution 2: LQC includes a generation strategy for steering test cases toward style updates that are likely to trigger layout under-invalidation bugs. The system focuses on CSS properties and mutations that should force layout recomputation, increasing the chance that a missed invalidation decision becomes

observable.

C. Challenge 3: Reducing Report Complexity

Test cases can contain many unrelated elements, declarations, and mutations. Likely only a small portion of the page causes a bug thus the original report may obscure the relevant DOM structure and CSS transition. The output can also contain many reports that are syntactically different but match a previously observed pattern. Without reduction and grouping, developers must repeatedly inspect large, redundant reports. At the same time fuzzers commonly generate many HTML and CSS variations that exercise the same underlying defect, so treating every report as a new bug would overstate the diversity of the findings. However, merging reports too aggressively could hide genuinely different bugs.

Preserving the bug trigger: Minimization is difficult because under-invalidation bugs depend on a precise sequence of layout states. The browser first lays out the original page, then applies a style update, and then decides which cached layout information can be reused. If the minimizer removes an element, style, or mutation that seems unrelated but changes this invalidation path, the bug may disappear even though the original conditions were present.

Separating base style state and modified style state: To identify which declarations are essential to the transition, LQC moves one modified style declaration at a time into the base style state. If the mismatch persists after the move, that declaration was not required as an incremental update and can be removed from the modified state. If the mismatch disappears, the declaration remains modified because changing it after the initial layout is part of the bug trigger.

Avoiding redundancy: In addition full minimization requires many browser executions. Before minimizing a new report, LQC can compare it with existing structural group rules. If it matches a group, then the system can assume that there is a common pattern and can retain the shared rule and minimized representative rather than fully minimizing an additional variation. Unmatched reports are minimized and can establish new groups.

Identifying repeated patterns: To determine whether reports represent recurring behavior or potentially new bugs, LQC records the DOM structure, base style state, modified style state, and affected element relationships. It uses this representation to group reports that match an existing structural rule, while preserving unmatched reports as separate candidates. The resulting groups are evidence of recurring patterns, not proof of a shared root cause; similarly, unmatched reports are candidates for separate validation rather than confirmed bugs.

Solution 3: LQC minimizes reports while preserving the state transition that exposes the bug, then groups structurally equivalent reports around a shared representative. This reduces the HTML and CSS a developer must inspect, avoids spending full minimization cost on reports already represented by a group, and separates repeated variants from potentially distinct candidates. The resulting reports are then suitable for browser-developer review, which provides the final evidence that a candidate is real and novel.

IV. Implementation

LQC is implemented in Python as an end-to-end, feedback-free fuzzing pipeline for browser layout invalidation. The implementation executes the serialized page in a real browser through Selenium WebDriver, and identifies an under-invalidation failure when the incremental layout version and refreshed layout version disagree. The same pipeline then generates candidate reports, minimizes them, and groups structurally related reports. The following sections describe the implementation: the layout oracle, the generation of web pages and bug candidates, and the clustering of reports.

Fig. 2 summarizes the main stages of the LQC implementation pipeline.

A. Layout Oracle

LQC checks layout correctness using self-differential testing: it tests the browser against itself rather than comparing one browser against another browser or against an external reference implementation. This is the core idea of the oracle. For each test case, LQC asks whether the browser produces the same layout for the same final page state when that state is reached in two different ways.

This differs from most existing browser fuzzers because LQC does not treat crashes, security violations, API exceptions, or cross-browser differences as its main correctness signal. Crash-oriented fuzzers can find memory-safety and stability bugs, but an under-invalidation bug may leave the browser running normally while still displaying a stale layout. Cross-browser rendering fuzzers can find cases where two browsers disagree, but disagreement alone does not show which browser is wrong or whether the problem is caused by incremental layout invalidation. In contrast, LQC compares two layout computations inside the same browser for the same final page state, making the oracle specifically targeted at stale incremental layout version results.

The first execution path is the incremental layout version. LQC loads a test case, applies a style update, and records the resulting layout. This is the point where the layout engine decides which cached layout state must be invalidated after the page changes. The second execution path is a refreshed layout version. After the style update has been applied, LQC forces the same browser to rebuild the page from scratch and compute the layout again from the same final HTML and CSS state creating the correct build of this web page configuration.

These two executions should produce identical results. If the incremental layout version differs from the refreshed layout version, then the browser has produced two different layouts for the same final page state. Because both results come from the same browser, the mismatch is not caused by differences between browser engines, feature support, or rendering policies. Instead, it is evidence that the incremental

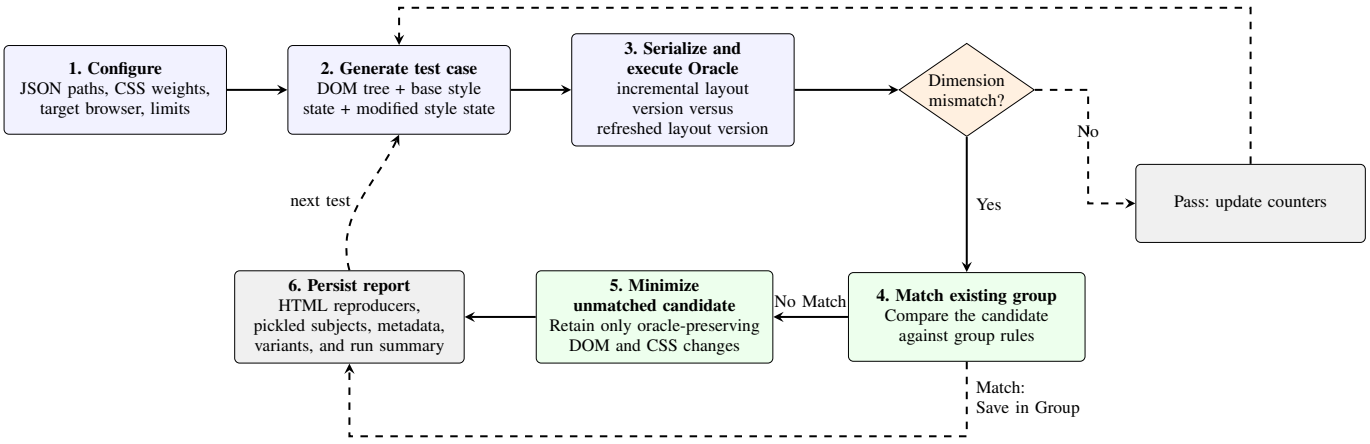

Fig. 2. LQC pipeline. Each test case is tested by a layout oracle. Passing subjects advance the fuzzing loop; candidates are structurally classified, minimized when necessary, and persisted with their reproduction artifacts and cumulative metrics.

layout version reused stale layout information and failed to invalidate part of the page correctly.

In practice, the oracle records the position and size of each element after the incremental update and then records the same information again after the refreshed layout version. It compares the position, width, and height for matching elements. If all recorded values match, the test passes. If any element has a different layout between the two paths, LQC reports the test as a layout-bug candidate.

This oracle is important because it gives LQC a correctness signal for bugs that do not cause crashes and do not require another browser to serve as a reference. By comparing the browser's incremental layout version against its own refreshed layout version, LQC directly targets the under-invalidation errors that conventional fuzzers are not designed to detect.

B. Test Case and Bug Generation

LQC generates test cases that are designed to exercise layout invalidation behavior rather than arbitrary browser behavior. Each test case contains an initial page state and a style update.

Internally, each test case is represented as a `RunSubject`:

- an `ElementTree`, which is a nested representation of the generated DOM;
- a base `StyleMap`, containing declarations applied in the initial document; and
- a modified `StyleMap`, containing declarations installed later by JavaScript.

The `ElementTree` defines the page structure, while the two `StyleMaps` define the base style state and the modified style state. The base style state determines the layout that the browser computes when the page first loads. The modified style state is applied later through JavaScript, forcing the browser to use its incremental layout invalidation engine. This test case produces the incremental layout version, which is then later compared to a refreshed layout version of the same final page state.

The generator builds pages from nested elements and text. Nested elements are important because many layout bugs are triggered when their are changes between parents, children, siblings, or descendants. Text is also useful because it introduces content-dependent sizing and positioning. Together, these structures create pages where a single style change can propagate through multiple parts of the layout tree, increasing the chance that a missed invalidation decision becomes observable.

To further increase the likelihood of triggering under-invalidation behavior, LQC biases generation toward CSS properties that are likely to affect layout geometry. Properties such as `display`, grid-related properties, `width`, `height`, percentage sizes, pixel sizes, and transforms are useful because they can change element size, position, layout mode, or parent-child layout relationships. These properties are more likely to require recomputation than purely visual properties, so they give the oracle more meaningful state changes to check.

The generator also supports weighted property selection. These weights allow an experiment to increase the probability of selecting properties that are expected to stress layout invalidation, while reducing or disabling properties that mostly add noise. This changes the distribution of test cases so that more executions target the kinds of mutations that are likely to expose stale layout results.

This design addresses the second challenge by making generation invalidation-aware. Rather than producing only valid HTML and CSS, LQC produces pages with layout-affecting style updates. As a result, each test case is more likely to exercise the browser's incremental layout logic and more likely to produce a bug candidate when that logic fails to invalidate stale geometry.

C. Minimization and Structural Clustering

Candidate pages are usually much larger than the condition required to reproduce a mismatch. LQC applies delta debugging to reduce a bug-triggering test case while preserving the oracle result. For each proposed simplification, it deep-

copies the current subject, executes the candidate in the target browser, and retains the copy only if the result remains a bug. Every accepted reduction becomes the new subject, so later reductions are tested against the smallest subject found so far. The delta-debugging process uses the following proposal families in order:

1) remove each element, including its associated base and modified style entries;
2) remove every style map for one element at a time;
3) remove individual base style state or modified style state declarations;
4) move one modified declaration into the base map, thereby testing whether that declaration must be an incremental change rather than merely being present in the final state;
5) apply readability-oriented transformations, including minimum box sizes, background colors, short identifiers, and an optional in-page control overlay.

The fourth step, implemented by `Minify_MoveStyleChangesToFirstLoad`, is particularly important for this bug class. If moving a declaration to the initial state preserves the mismatch, it is not necessary for the incremental transition and can be removed from the set of modified properties. Conversely, declarations that cannot be moved identify the state change that is more likely to matter to invalidation. The final oracle run is recorded again after all proposal families have been exhausted. If the candidate no longer reproduces, it is counted as a false positive rather than saved as a confirmed layout report. A layout mismatch with an empty modified-style map is also counted separately because it no longer expresses the intended incremental CSS-change scenario.

In order to cluster bugs the runner first constructs a small repository of known-safe subjects. It generates and executes subjects until the configured safe directory contains 25 passing serialized `RunSubject` objects. These safe samples are used to prevent the grouping algorithm from accepting a rule that would also match known non-bug inputs.

Full reduction is expensive when many test cases trigger a pattern already represented in the report directory. With sorting enabled, LQC therefore attempts to match a new candidate against existing bug groups before full minimization. The `--no-sort` option disables structural classification and grouping, causing each candidate to be fully minimized and saved as an individual report rather than compared with existing bug groups.

A test case is represented as a tree in which each node records base style state, modified style state, and children; text is represented explicitly as a node in the tree. The first node with modified style state is used as the start node for a rule because it anchors comparison relative to this modified node. The rule represents the DOM structure relative to this anchor: it compares the modified node and its ancestor/descendant structure, rather than depending on that node's absolute location in the full page tree. This allows structurally similar update patterns to match even when they occur in different test cases.

TABLE I
BROWSER EVALUATION BENCHMARKS.

Browser	Rendering Engine	Driver
Chrome	Blink	ChromeDriver
Firefox	Gecko	GeckoDriver

A group stores a merged representative tree and an extracted JSON rule. To form a candidate rule, the implementation merges two trees recursively. Equal tags, values, and declarations are retained. Disagreements are represented by the wildcard value `diff`. The resulting rule consists of an ordered HTML pattern and the common base style state and modified style state requirements at the start node. Matching checks both components: the incoming subject must contain the ordered structural pattern and at least one matching element must have the required base style state and modified style state declarations.

Before accepting either a match or a new group, the rule is validated against the safe repository. Rules with no modified-style requirement or with a trivial single-element pattern are rejected, and a rule is rejected if it matches any safe sample. This is a conservative guard against a broad structural description that would classify ordinary pages as bug instances. It makes group promotion depend on both bug examples and known non-bug examples.

If an incoming candidate matches an existing rule, the runner skips the full reduction and saves it as another instance under that group. Otherwise, it minimizes the candidate first. The final minimized subject is then compared against standalone reports. Matching a standalone report promotes the two reports into a newly created `bug-group-*` directory and writes the extracted rule. Group artifacts are recomputed so the merged representative and rule correspond to the instances currently stored in that directory. The sorting time is measured separately and added to the run summary alongside minimization time; this makes the cost and benefit of grouping measurable in Section V. Each saved report is placed in either a standalone `bug-*` directory or a child directory of a `bug-group-*` directory.

V. EVALUATION

Our evaluation of LQC is guided by these fundamental questions:

RQ1: Is LQC at triggering an under-invalidation bugs?
RQ2: Do minimization and clustering reduce report complexity?
RQ3: Does LQC find confirmed under-invalidation bugs?

Benchmarks: Table I summarizes the browser targets evaluated by LQC. Our evaluation uses Chromium, which exercises the Blink layout engine, and Firefox, which exercises the Gecko layout engine. LQC uses ChromeDriver and GeckoDriver, respectively.

TABLE II
BUG-DISCOVERY RESULTS ACROSS BROWSER AND CONFIGURATIONS.

Browser	No weights			Weights		
	Speed	Bugs	Rate	Speed	Bugs	Rate
Chromium	196.8	105	0.889%	558.0	100	0.299%
Firefox	495.9	63	0.212%	492.8	137	0.463%

Note: Runs use a 60-minute execution window. Speed is measured in executed tests per minute; rate is bugs divided by executed tests.

Experiment Setup: Because no existing fuzzer specifically targets under-invalidation bugs, our evaluation focuses on LQC 's efficiency and quality of the results it produces. For RQ1, we run each browser with and without property weights for 60 minutes. We save several characteristics of these runs including speed(number of tests ran per minute), bugs found, rate of bug discovery, and the overall execution time. During the 60 minute window every minute a snapshot of these metrics were saved to accurately track the metrics across time. This data is represented in Fig. 3, which shows the bugs detected over execution time, and Table II, which showcases metrics at the end of the 60 minute window. For RQ2, the evaluations compares Firefox runs with sorting enabled and disabled over a nine hour execution. We measure the number of css properties in the original reports, minimized reports and clustered reports, cumulative post-processing time of minimization and clustering, and execution time. Measuring total css properties takes into account only the bugs that are within a bug group as to not inflate the total css properties of grouped or minimized bugs that were never grouped. For RQ3, we report the status of confirmed bugs and use report groups and single bugs to describe the diversity of the report repository.

A. RQ1: Finding Under-Invalidation Reports

RQ1 establishes whether LQC can repeatedly drive independent browser engines into under-invalidation behavior. Property weighting is a secondary comparison: it biases generation toward selected CSS properties, while the unweighted configuration samples from the same property space without that bias. Fig. 3 shows that reports continue to accumulate during the 60-minute window, and Table II distinguishes report count from throughput and report-triggering rate.

Outcomes: Every configuration finds dozens of reports within 60 minutes: Chromium finds 105 reports without property weights and 100 reports with property weights, while Firefox finds 63 and 137 reports, respectively. The nonzero rates in both Blink and Gecko show that LQC reaches the target behavior in two independent layout engines, rather than depending on one browser-specific implementation.

The weighted comparison explains how configuration choice affects that ability. In Firefox, weighting raises the report count from 63 to 137 and the report-triggering rate from 0.212% to 0.463%, while throughput remains nearly unchanged (495.9 versus 492.8 tests per minute). In Chromium, weighting increases throughput but does not improve discovery rate: the unweighted configuration produces more reports (105 versus 100) and a higher report-triggering rate (0.889% versus 0.299%). Thus, higher throughput alone does not explain the usefulness of property weights depends on the browser engine and the properties they emphasize.

RQ1: LQC finds under-invalidation reports in both Chromium and Firefox within one hour. Property weights improves discovery rate, but their benefit is browser-dependent rather than universal.

B. RQ2: Reducing Developer Triage Work

Finding a bug is most useful when it is easy to understand and analyze. RQ2 asks whether minimization and clustering reduce that burden, in addition it is important to examine the effeciency impacts. Minimization removes unnecessary CSS style elements from a report. Clustering recognizes repeated reports and retains a shared rule and minimized representative, which can in turn be used to avoid full minimization for every bug found. We therefore evaluate two linked outcomes: how much smaller the developer-facing reports become, and whether the extra processing reduces fuzzing throughput. Fig. 4 measures the former; Fig. 5 measures the latter.

Outcomes: The report-size results show that the clustering and minimization pipeline meaningfully changes the volume of a bug analysis needed. Minimization reduces the style elements in grouped reports by 62.60%relative to the original reports. Combining minimization with clustering reduces the style-element volume by 99.74% relative to originals and by 99.32% relative to minimized reports. Clustering removes a significant amount of repeated work across reports that represent the same behavior. LQC retains the group's shared rule and minimized representative instead of independently minimizing and inspecting the new report. Developers can therefore focus on one representative report per behavior rather than repeatedly examining equivalent reports.

In addition the minimization-and-sorting pipeline accounts for only 14.16% of the Firefox sorting run's runtime. Moreover, specifically the sorting of bug reports into bug groups does not introduce an overhead; instead, it reduces post-processing cost when compared to simply minimizing each bug. At the latest common snapshot, the sorting configuration executes $\sim$130,640 test cases, compared with $\sim$126,210 without sorting, while spending $\sim$4,550 seconds on minimization and sorting rather than $\sim$5,890 seconds without sorting. When a report matches an existing group, LQC can reuse that group's representative instead of fully minimizing the new report. Consequently, sorting reduces cumulative post-processing time by $\sim$22.6% and enables $\sim$4,430 additional tests to run.

RQ2: Minimization and clustering make reports substantially easier to dissect while preserving, and in this experiment improving, fuzzing throughput. They reduce both the size of individual reports and the repeated work spent on reports already represented by a bug group.

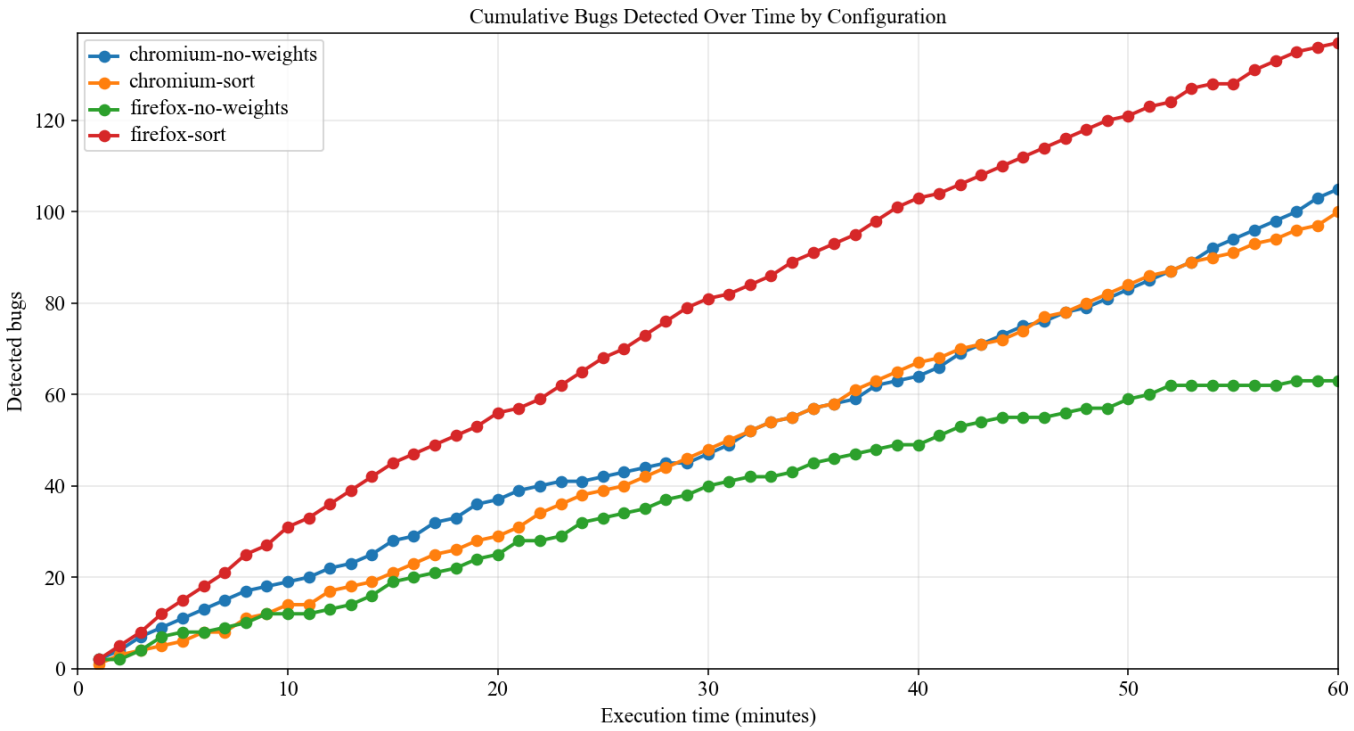

Fig. 3. Cumulative bug reports for the weighted and no-weight configurations over the 60-minute evaluation window.

C. RQ3: Finding Confirmed Bugs

The candidate reports measured in RQ1 are not, by themselves, evidence of distinct root-cause bugs. RQ3 therefore asks whether LQC produces reports that survive external developer review. LQC produced 16 confirmed under-invalidation bugs; Table III records their current statuses. 7 of these reports were fixed or closed by developers. We also reproduce 7 human-submitted Mozilla bugs, showing that the oracle can reach known under-invalidation behavior.

Clustering provides complementary evidence about report diversity before manual validation. Fig. 7 shows how reports are divided into bug groups and single bugs over time, while Fig. 6 shows the final size distribution of the Firefox groups. **Outcomes:** Table III establishes that LQC's reports are actionable beyond the experimental oracle. It produced 16 confirmed bugs, and 7 were fixed or closed. These outcomes provide external evidence that the tool reaches real under-invalidation defects rather than only reporting test artifacts.

The clustering results provide evidence that LQC discovers a diverse set of potential bugs rather than merely generating many syntactic variations of the same behavior. The clustering process groups reports with shared behavior while retaining a minimized representative for each group. At the latest common snapshot, the Firefox sorting run produces 565 reports across 46 groups and 37 unmatched reports. The number of groups, each represented by distinct HTML and CSS combinations, indicates that the report repository contains substantial behavioral variation rather than a single bug expressed repeatedly.

TABLE III
STATUS OF BUG REPORTS SUBMITTED TO CHROME AND FIREFOX.

Browser	Bug ID	Brief description	Status
Chrome	1137427	Repeated white-space application failure	Fixed
Chrome	1166887	Dynamic table-caption vertical order swap	Fixed
Chrome	1189755	Fixed-position grid layout under-invalidation	Fixed
Chrome	1189762	None-to-inline-grid display invalidation	Open
Chrome	1190220	Grid layout page crash	Fixed
Chrome	1219641	Inline-grid list-item text-indent invalidation	Open
Chrome	1224721	Grid layout under-invalidation report	Fixed
Chrome	1229681	Grid child display-none invalidation	Open
Firefox	1721719	Table row-group height inconsistency	Open
Firefox	1724982	Table min-height overflow discrepancy	Open
Firefox	1724991	Dynamic break-spaces invalidation inconsistency	Open
Firefox	1733276	Display-inline margin line-wrapping inconsistency	Open
Firefox	1735376	Grid table-row height inconsistency	Closed
Firefox	1735931	Percent-sized grid-item height inconsistency	Closed
Firefox	1748891	Sticky padding scrollport position inconsistency	Open
Firefox	1724999	Layout Quick Check metabug	Open

These counts are evidence of potential diversity, not confirmed root-cause-bug counts; confirmation of novelty comes from the submitted reports reviewed by browser developers in Table III.

RQ3: LQC finds confirmed under-invalidation bugs: it produced 16 confirmed bugs, of which 7 were fixed or closed. Clustering helps further separate recurring reports from candidates to make independent validation easier.

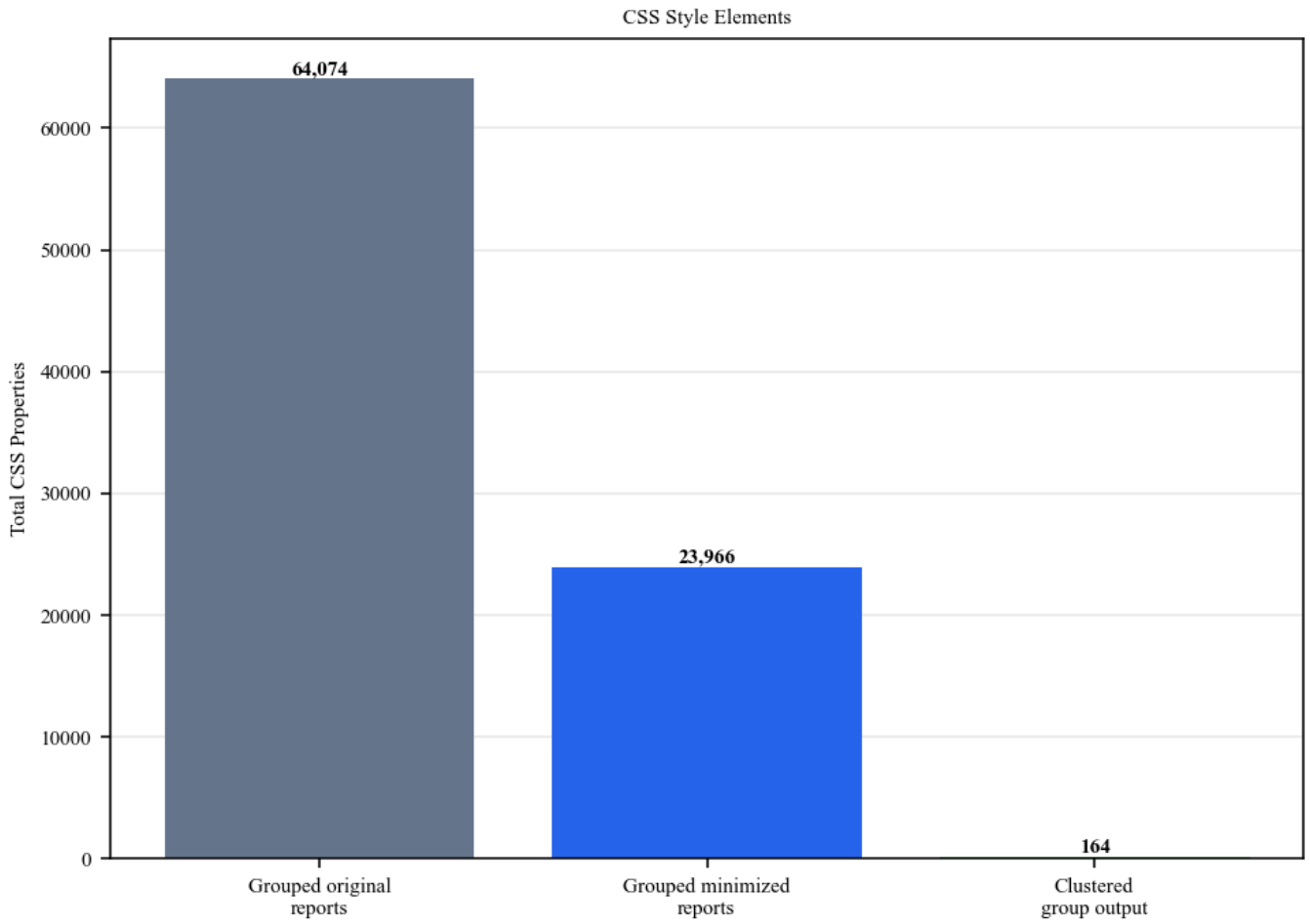

Fig. 4. Total CSS style elements in grouped Firefox reports before minimization, after minimization, and after clustering.

VI. Discussion & Threats to Validity

A. Optimizing Runtime Throughput

Several parts of the workflow can limit throughput, particularly test generation, minimization, and sorting. More efficient minimization and sorting algorithms could reduce this overhead. For example, minimization could use a divide-and-conquer strategy that first attempts to remove groups of HTML nodes or CSS declarations, then progressively tests smaller groups when necessary. Unlike the current element-by-element and declaration-by-declaration approach, this could reduce the number of browser executions required to find a compact subject.

Test generation could be improved by detecting and discarding semantically redundant mutations or combinations, such as style changes that produce the same computed value or cannot affect the generated element's layout. This would allow more browser executions to target meaningful layout interactions.

Finally, generation, minimization, and sorting can be parallelized across multiple browser instances or machines. Candidate bugs could be queued for minimization and sorting while other workers continue generating and executing new tests. This would increase throughput, although it introduces resource-management, synchronization, and duplicate-reporting challenges.

Despite these throughput limitations, the current workflow remains effective because it successfully generates, detects, minimizes, and organizes reproducible browser-layout bugs.

B. Supporting Other Browsers

Supporting a browser requires a browser-specific configuration, a compatible automation driver, and validation that the browser exposes layout information consistently enough for the test oracle. These requirements make it straightforward to support widely used browsers with robust WebDriver implementations, but they limit practical testing of uncommon browsers and older browser versions.

Less frequently tested browsers may contain defects that are not found by testing dominant browser engines. Supporting these browsers could therefore reveal bugs in engines that receive less testing and maintenance. However, support for additional browsers has diminishing returns because most web users are concentrated in a small number of major browser engines.

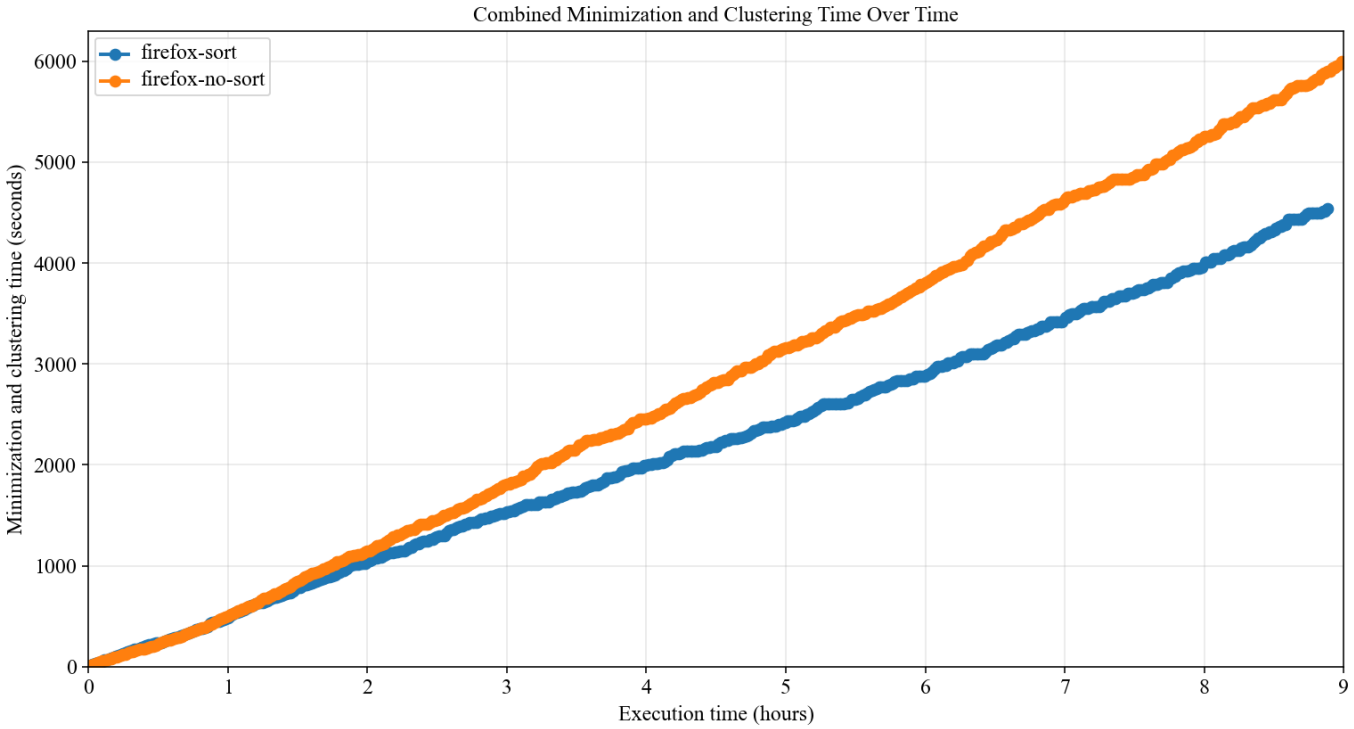

Fig. 5. Cumulative minimization and clustering time for Firefox sorting and no-sorting configurations.

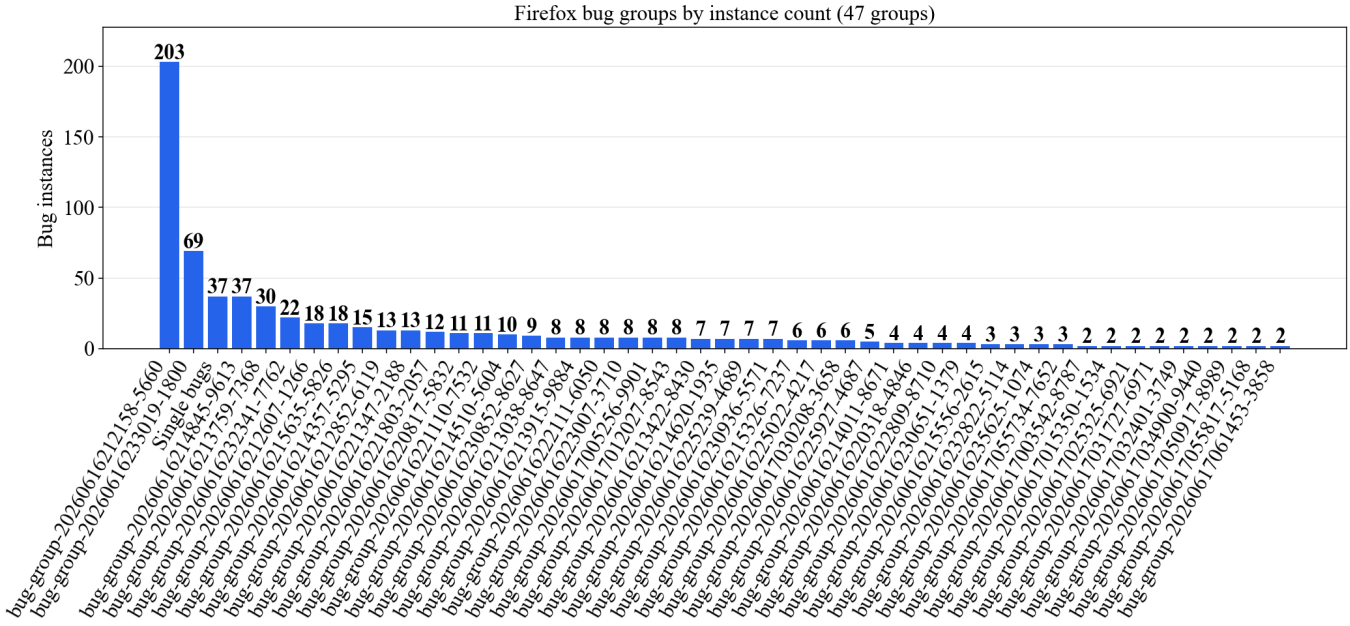

Fig. 6. Firefox bug-group sizes, with all single bugs combined into one bar.

The supported Chromium, WebKit/Safari, and Firefox configurations cover most browser usage. Worldwide in May 2026, Chrome accounted for 70.25% of browser usage, Safari for 15.72%, and Firefox for 5.14%; together, these browsers account for 91.11% of usage [31]. This leaves only 8.89% of usage outside of the main target of LQC.

Nevertheless, the current implementation remains effective because it supports major browser engines with mature automation tooling and substantial user populations.

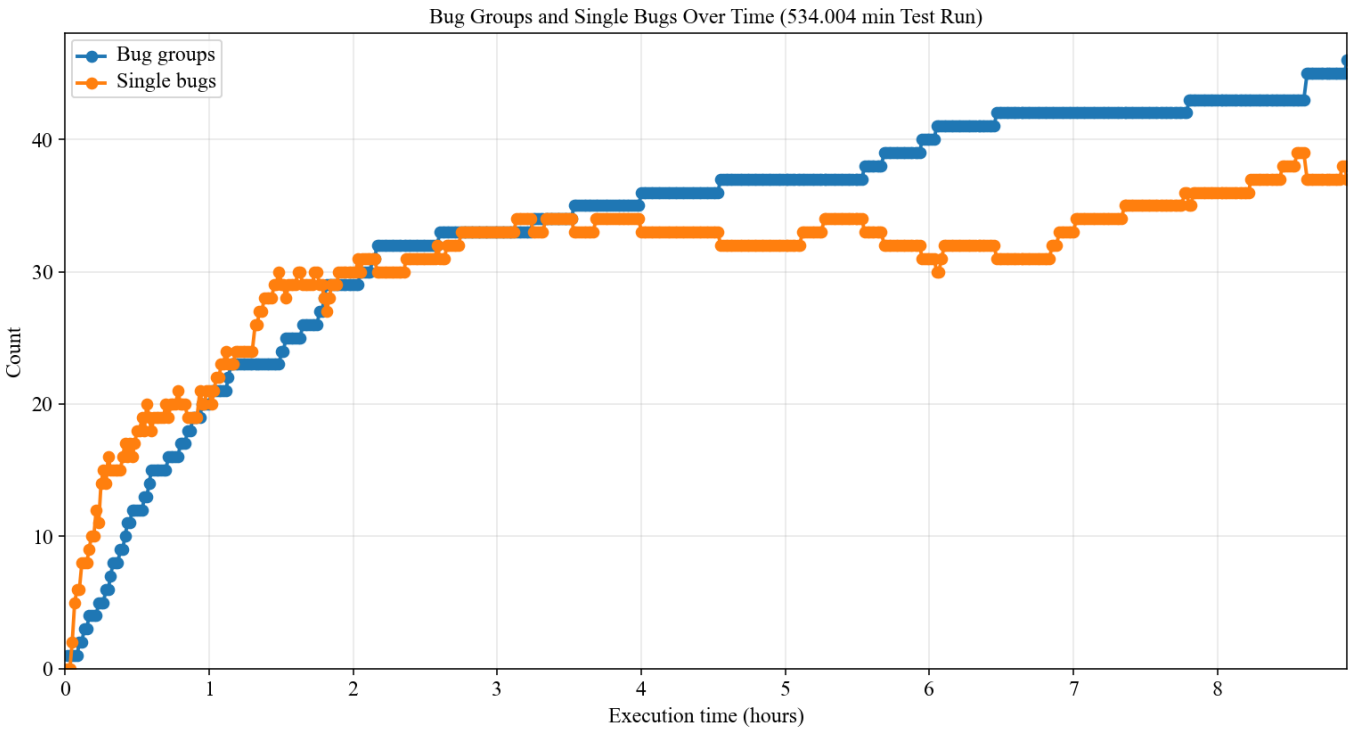

Fig. 7. Bug-group and single-bug counts by total detected reports in the Firefox sorting run.

C. Increasing Bug Rate

The bug detection rate directly affects the efficiency of the tool, since a higher rate produces more candidate bugs per executed test case. However, repeatedly finding the same underlying defect is less useful than discovering distinct bugs. Nevertheless, the tool remains effective because sorting and grouping reduce duplicate reports and help distinguish recurring bug patterns.

One approach to increasing the bug rate is to identify a stronger static style-weight configuration before a testing run begins. Results from earlier runs can identify HTML structures, CSS properties, and style updates that have produced layout inconsistencies most often. These observations can then be used to assign greater initial weights to promising properties while preserving lower probabilities for less frequently selected properties.

Separately, a feedback-driven system could dynamically adjust style-selection probabilities while a testing run is running. It could increase the weights of properties associated with novel layout discrepancies and reduce the weights of configurations that repeatedly produce uninteresting or duplicate results. The system should retain a degree of random exploration so that it does not become overly focused on known bug patterns. Nevertheless, the current random generation strategy remains useful because it can discover unexpected bug classes without requiring prior knowledge of effective configurations.

VII. Conclusion

Acknowledgments

References

[1] F. Montenegro, “The state of workforce security: Key insights for it and security leaders,” Palo Alto Networks, Tech. Rep., Jan. 2025.

[2] S. Song, J. Hur, S. Kim, P. Rogers, and B. Lee, “R2Z2: Detecting rendering regressions in web browsers through differential fuzz testing,” in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE ’22. ACM, 2022.

[3] C. Zhou, Q. Zhang, B. Qian, and Y. Jiang, “JANUS: Detecting rendering bugs in web browsers via visual delta consistency,” in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, ser. ICSE ’25. IEEE, 2025.

[4] S. Song and B. Lee, “Metamong: Detecting render-update bugs in web browsers through fuzzing,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE ’23. ACM, 2023.

[5] Chromium, “Issue 497183437,” Chromium Issue Tracker. [Online]. Available: https://issues.chromium.org/issues/497183437

[6] ——, “Issue 379886422 resources,” Chromium Issue Tracker. [Online]. Available: https://issues.chromium.org/issues/379886422/resources

[7] Mozilla, “Bug 1452426,” Mozilla Bugzilla. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=1452426

[8] ——, “Bug 539356,” Mozilla Bugzilla. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=539356

[9] WebKit, “Bug 233421,” WebKit Bugzilla. [Online]. Available: https://bugs.webkit.org/show_bug.cgi?id=233421

[10] ——, “Bug 97801,” WebKit Bugzilla. [Online]. Available: https://www2.webkit.org/show_bug.cgi?id=97801

[11] I. Kilpatrick and K. Ishii, “RenderingNG deep-dive: LayoutNG,” Chrome for Developers, 2021, last updated October 8, 2021. [Online]. Available: https://developer.chrome.com/docs/chromium/layoutng?hl=en

[12] MozillaWiki, “Gecko: Reflow refactoring,” MozillaWiki, 2006, landed December 7, 2006. [Online]. Available: https://wiki.mozilla.org/Gecko:Reflow_Refactoring

[13] The Chromium Projects, “Blink: Rendering engine,” The Chromium Projects, accessed June 2, 2026. [Online]. Available: https://www.chromium.org/blink/
[14] Mozilla, “Gecko,” Firefox Source Docs, accessed June 2, 2026. [Online]. Available: https://firefox-source-docs.mozilla.org/overview/gecko.html
[15] WebKit, “Webkit,” WebKit, accessed June 2, 2026. [Online]. Available: https://webkit.org/
[16] Open Web Advocacy, “Break google’s search monopoly without breaking the web,” Open Web Advocacy, Apr. 2025, published April 23, 2025. [Online]. Available: https://open-web-advocacy.org/blog/break-googles-search-monopoly-without-breaking-the-web/
[17] M. Kosaka, “Inside look at modern web browser, part 3,” Chrome for Developers, 2018. [Online]. Available: https://developer.chrome.com/blog/inside-browser-part3
[18] MDN Web Docs, “Web performance,” MDN Web Docs, accessed June 2, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Performance
[19] P. Panchekha and C. Harrelson, *Reusing Previous Computations*. Oxford University Press, 2024, pp. 443–494. [Online]. Available: https://browser.engineering/invalidation.html
[20] Mozilla, “Performance best practices for firefox front-end engineers,” Firefox Source Docs, accessed June 2, 2026. [Online]. Available: https://firefox-source-docs.mozilla.org/performance/bestpractices.html
[21] ——, “Fuzzing,” Firefox Source Docs, accessed June 2, 2026. [Online]. Available: https://firefox-source-docs.mozilla.org/tools/fuzzing/index.html
[22] P. Ventuzelo, “Practical web browser fuzzing,” Ringzer0 Training, 2023. [Online]. Available: https://ringzer0.training/practical-web-browser-fuzzing/
[23] I. Araoz Severiche, “Web fuzzing: A practical testing methodology,” Medium, Feb. 2026, published February 7, 2026. [Online]. Available: https://iaraoz.medium.com/web-fuzzing-a-practical-testing-methodology-1f24198e2ddc
[24] H. Han, D. Oh, and S. K. Cha, “CodeAlchemist: Semantics-aware code generation to find vulnerabilities in JavaScript engines,” in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS ’19, 2019.
[25] S. Park, W. Xu, I. Yun, D. Jang, and T. Kim, “Fuzzing JavaScript engines with aspect-preserving mutation,” in *Proceedings of the IEEE Symposium on Security and Privacy*, 2020.
[26] H. Peng, Z. Yao, A. Amiri Sani, D. J. Tian, and M. Payer, “GLeeFuzz: Fuzzing WebGL through error message guided mutation,” in *Proceedings of the USENIX Security Symposium*, 2023.
[27] O. Draissi, T. Cloosters, D. Klein, M. Rodler, M. Musch, M. Johns, and L. Davi, “Wemby’s web: Hunting for memory corruption in WebAssembly,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, 2025.
[28] Y. Zeng, Y. Wu, X. Lu, and C. Zhang, “RTFuzz: Fuzzing browsers via efficient render tree mutation,” *Computers & Security*, vol. 161, p. 104756, 2026.
[29] J. Liu, Y. Chen, E. Atkinson, Y. Feng, and R. Bodik, “Conflict-driven synthesis for layout engines,” *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 132:1–132:22, 2023.
[30] S. Kim, Y. M. Kim, J. Hur, S. Song, G. Lee, and B. Lee, “FuzzOrigin: Detecting UXSS vulnerabilities in browsers through origin fuzzing,” in *Proceedings of the 31st USENIX Security Symposium*, ser. USENIX Security ’22. USENIX Association, 2022. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/kim
[31] StatCounter GlobalStats, “Browser market share worldwide,” StatCounter GlobalStats, May 2026, worldwide browser market share, May 2026; accessed June 24, 2026. [Online]. Available: https://gs.statcounter.com/browser-market-share